EcoTrack API · DevOps

Pipeline CI/CD, containerização e orquestração
de uma API ESG em Java/Spring Boot + Oracle

Aluno	Paulo Souza
RM	566228
Projeto de origem	EcoTrack API (Java 21, Spring Boot 3.3.5, Oracle)
Ferramenta de CI/CD	GitHub Actions
Orquestração	Docker Compose
Repositório	github.com/psouzah12/ecotrack-devops
Data	Outubro de 2026

1. Visão geral

O EcoTrack é uma API REST que centraliza dados ambientais de empresas: empresas, indicadores (água, energia, resíduos, CO2), ações ESG e relatórios. Esta entrega pega o projeto Java/Spring da sprint anterior e integra práticas de DevOps para automatizar o ciclo de vida completo, do commit até a produção.

Repositório: https://github.com/psouzah12/ecotrack-devops
Execução documentada: https://github.com/psouzah12/ecotrack-devops/actions/runs/38489083683

Objetivo	Como foi atendido
Pipeline CI/CD com build, testes e deploy	GitHub Actions com 6 jobs: build → testes unitários + testes de integração → imagem Docker no GHCR → deploy em staging → deploy em produção
Containerizar a aplicação	Dockerfile multi-stage, JRE 21 Alpine, usuário sem root, jar em camadas, healthcheck, imagem amd64 + arm64
Orquestrar a infraestrutura	docker-compose.yml com API + Oracle, redes frontend/backend (interna), volumes de dados e logs, variáveis por ambiente
Simular um ambiente de produção	Profiles staging/prod, actuator (health e info), segredos fora do código, deploy com rollback, smoke tests, banco de produção sem dados de teste

Melhorias feitas na aplicação

- **Actuator:** `/actuator/health` (liveness e readiness com o banco) e `/actuator/info` (versão, commit e ambiente). Usados pelo healthcheck do Docker e pelos smoke tests.
- **Profiles por ambiente:** `staging` e `prod`, com log em arquivo (com rotação) e nível de log próprio. Em produção o Flyway não aplica os dados de exemplo (`db/seed`).
- **Credenciais fora do código:** os usuários da API, antes fixos no `SecurityConfig`, agora vêm de variáveis de ambiente.
- **Erros corretos:** JSON inválido, parâmetro com tipo errado, rota inexistente e método não suportado devolviam 500; agora devolvem 400, 404 e 405. O erro inesperado passou a ser registrado em log.
- **Transações:** `open-in-view` desligado e `@Transactional` nos services.
- **Página de status** em `/` mostrando o ambiente, a saúde da API e do banco, a versão, o commit e o resumo ESG. O Swagger mostra o ambiente no título.
- **Testes:** de 2 testes triviais para 56 testes (49 unitários/web + 7 de integração com Oracle real).

2. Pipeline CI/CD

Ferramenta: GitHub Actions, arquivo `.github/workflows/ci-cd.yml`. **Registry:** GitHub Container Registry (GHCR), autenticado com o `GITHUB_TOKEN` do próprio run.



Job	O que faz
1 · Build	Checkout, JDK 21 (Temurin) com cache do Maven, `mvn package`; publica o `ecotrack-api.jar` como artefato.
2 · Testes unitários	`mvn test`: 49 testes de services (Mockito) e controllers com a segurança real (MockMvc). O JaCoCo reprova o job se a cobertura de linhas ficar abaixo de 80%. Gera uma tabela na aba Summary e publica os relatórios.
3 · Testes de integração	`mvn verify -DskipUnitTests`: Testcontainers sobe um Oracle Free, o Flyway aplica as migrations e a API é testada via HTTP (CRUD, 401, 409 por chave estrangeira, health, info).
4 · Imagem Docker	Buildx + QEMU geram a imagem para `linux/amd64` e `linux/arm64`, publicada no GHCR com as tags `sha-<commit>`, `1.1.0` e `latest` / `develop`. O cache de camadas fica no GitHub (`type=gha`).
5 · Deploy staging	Environment `staging`: login no GHCR, `scripts/deploy.sh staging <imagem>` (Docker Compose) e `scripts/smoke-test.sh`.
6 · Deploy produção	Environment `production`, só no branch `main` e depois de staging aprovado. Com *Required reviewers*, o job espera aprovação manual. Usa a mesma imagem e os mesmos scripts.

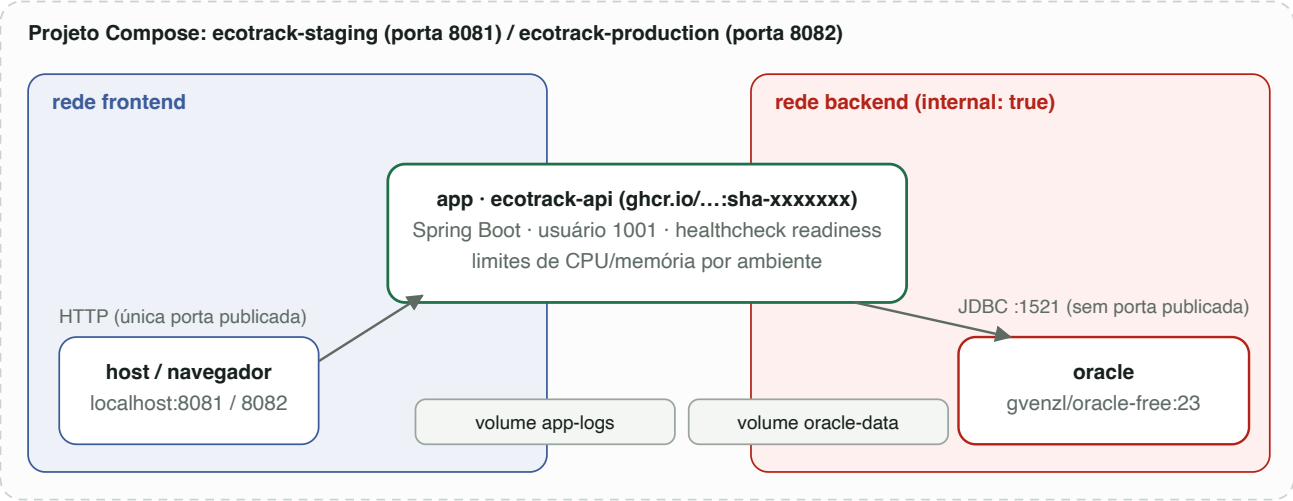
Lógica e decisões

- **Gatilhos:** PR para `main` roda só build e testes; push em `develop` vai até staging; push em `main` vai até produção. O workflow também pode ser disparado manualmente.
- **Build once, deploy many:** a imagem é gerada uma única vez e identificada pelo commit. Staging e produção recebem o mesmo artefato que passou nos testes.
- **Portões de qualidade:** a imagem só é publicada se as duas suítes passarem, e produção só recebe deploy se staging passar no smoke test.
- **Smoke test:** verifica health (API e Oracle), ambiente e versão no ar, leitura pública, bloqueio de escrita sem login e um ciclo real de escrita no banco (cria e apaga). O resultado vai para a aba Summary.

- **Rollback automático:** se a API não ficar *healthy*, o `deploy.sh` exibe os logs e volta para a imagem que estava rodando.
- **Segredos:** senhas vêm dos secrets de cada Environment do GitHub. Na ausência deles, o `deploy.sh` gera senhas fortes e as guarda no host (`~/.ecotrack`, permissão 600). Nada sensível fica no repositório.
- **Onde roda o deploy:** por padrão em runners `ubuntu-24.04`, que sobem o ambiente completo (Oracle + API) dentro da máquina do runner, validam e descartam no fim. Com a variável `DEPLOY_RUNNER=self-hosted`, os mesmos jobs fazem o deploy numa máquina própria, com ambientes permanentes.
- **Fila em vez de cancelamento** (`cancel-in-progress: false`): um deploy nunca é interrompido no meio. **Dependabot** abre PRs de atualização que passam pelo mesmo pipeline.

3. Docker

Arquitetura



Um único `docker-compose.yml` atende todos os ambientes. O que muda vem de variáveis: imagem, porta, profile, senhas e limites de recursos. Cada ambiente roda como um projeto Compose separado (`-p ecotrack-staging` / `-p ecotrack-production`), com containers, redes e volumes isolados. O Oracle fica só na rede `backend`, marcada como `internal`: não tem porta publicada nem saída para a internet. A API é a única porta de entrada.

Recurso	Uso
Serviços	`oracle` (gvenzl/oracle-free:23-slim) e `app`. A API espera o Oracle ficar *healthy* (`depends_on: service_healthy`).
Redes	`frontend` (porta publicada da API) e `backend` (`internal: true`, API + Oracle).
Volumes	`oracle-data` (dados do banco, preservados entre deploys) e `app-logs` (log da API em `/app/logs`).
Variáveis	`deploy/staging.env` e `deploy/production.env` (não secretas); secrets do GitHub para senhas; `.env.example` documenta todas.
Robustez	`restart: unless-stopped`, healthchecks, desligamento gracioso, limites de CPU/memória, rotação de logs do Docker.

Dockerfile e imagem criada

```
# syntax=docker/dockerfile:1
# ------------------------------------------------------------------------
# EcoTrack API - imagem de produção (multi-stage)
#   estágio 1 (build):   compila com Maven + JDK 21 e separa o jar em camadas
#   estágio 2 (runtime): só o JRE 21 Alpine + as camadas, rodando sem root
# ------------------------------------------------------------------------

# O build roda na arquitetura de quem está construindo ($BUILDPLATFORM).
# O jar é o mesmo para amd64 e arm64, então o Maven executa uma vez só
# mesmo quando o pipeline gera a imagem multi-arquitetura.
FROM --platform=$BUILDPLATFORM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /workspace

# 1) dependências primeiro: essa camada só é refeita quando o pom.xml muda
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

# 2) código-fonte e empacotamento (os testes já rodaram nos jobs anteriores do pipeline)
COPY src ./src
RUN mvn -B -ntp package -DskipTests \
 && java -Djarmode=tools -jar target/ecotrack-api.jar extract --layers --launcher --destination
target/extracted


FROM eclipse-temurin:21-jre-alpine AS runtime

# usuário sem privilégios + pasta de logs (montada como volume no compose)
RUN addgroup -S ecotrack && adduser -S -G ecotrack -u 1001 ecotrack \
 && mkdir -p /app/logs && chown -R ecotrack:ecotrack /app
WORKDIR /app

# camadas da menos para a mais volátil: num deploy comum só "application" muda
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/dependencies/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/spring-boot-loader/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/snapshot-dependencies/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/application/ ./

# metadados preenchidos pelo pipeline (docker build --build-arg ...)
ARG APP_VERSION=dev
ARG GIT_COMMIT=local
ARG BUILD_DATE=unknown
LABEL org.opencontainers.image.title="EcoTrack API" \
      org.opencontainers.image.description="API REST de indicadores e ações ESG" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.revision="${GIT_COMMIT}" \
      org.opencontainers.image.created="${BUILD_DATE}" \
      org.opencontainers.image.authors="Paulo Souza - RM566228"

ENV APP_COMMIT=${GIT_COMMIT} \
    JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError -
Duser.timezone=America/Sao_Paulo -Djava.security.egd=file:/dev/./urandom"

USER ecotrack
EXPOSE 8080

# o Docker marca o container como unhealthy se a API ou o banco caírem
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD wget -q -O /dev/null http://127.0.0.1:8080/actuator/health/readiness || exit 1

ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Item	Valor
Imagem	`ghcr.io/psouzah12/ecotrack-api`; tags `sha-809da10`, `1.1.0`, `latest` (pública)
Base	`eclipse-temurin:21-jre-alpine` (runtime) · `maven:3.9.9-eclipse-temurin-21` (build)
Plataformas	linux/amd64 e linux/arm64
Tamanho	408 MB (a imagem da sprint anterior tinha 707 MB)
Camadas da aplicação	dependências ≈ 64 MB · loader ≈ 4 KB · classes da aplicação ≈ 0,3 MB
Segurança	usuário `ecotrack` (uid 1001), sem root; contexto de build só com `pom.xml` e `src/`

Comandos usados

```
# ambiente local (build + API + Oracle)
docker compose up --build -d
docker compose ps
docker compose logs -f app
docker compose down            # -v apaga o volume do banco

# imagem
docker build -t ecotrack-api:local --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=$(git rev-
parse HEAD) .
docker image inspect ecotrack-api:local
docker history ecotrack-api:local

# pipeline (job 4): imagem multi-arquitetura no GHCR
docker buildx build --platform linux/amd64,linux/arm64 -t ghcr.io/psouzah12/ecotrack-api:sha-
809da10 --push .

# deploy de cada ambiente (jobs 5 e 6)
scripts/deploy.sh staging ghcr.io/psouzah12/ecotrack-api:sha-809da10
scripts/smoke-test.sh http://localhost:8081 staging
scripts/deploy.sh production ghcr.io/psouzah12/ecotrack-api:sha-809da10
scripts/smoke-test.sh http://localhost:8082 production

# por dentro do deploy.sh
docker compose --env-file deploy/staging.env -p ecotrack-staging -f docker-compose.yml pull
docker compose --env-file deploy/staging.env -p ecotrack-staging -f docker-compose.yml up -d --
wait --remove-orphans
```

02-docker-build — ecotrack-devops

```
$ docker build -t ecotrack-api:local --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=809da10084a5135c6ebf9403a0cc1571faf5e30e .
#1 [internal] load build definition from Dockerfile
#1 [internal] load build definition from Dockerfile
#3 CACHED
#4 [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-21
#5 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#4 [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-21
#6 [internal] load .dockerignore
#7 [build 1/6] FROM docker.io/library/maven:3.9.9-eclipse-temurin-21@sha256:3a4ab3276a087bf276f79cae96b1af04f53731bec53fb2e651aca79e4b10
211e
#8 [runtime 1/7] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141ce665ca3ea85e8b5cd648eafbc3c0c90dd79d6537684
e4555
#9 [internal] load build context
#10 [build 4/6] RUN mvn -B -ntp dependency:go-offline
#10 CACHED
#11 [runtime 4/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/dependencies/ ./
#11 CACHED
#12 [build 6/6] RUN mvn -B -ntp package -DskipTests  && java -Djarmode=tools -jar target/ecotrack-api.jar extract --layers --launcher --
destination target/extracted
#12 CACHED
#13 [runtime 6/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/snapshot-dependencies/ ./
#13 CACHED
#14 [runtime 5/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/spring-boot-loader/ ./
#14 CACHED
#15 [runtime 3/7] WORKDIR /app
#15 CACHED
#16 [build 2/6] WORKDIR /workspace
#16 CACHED
#17 [build 3/6] COPY pom.xml .
#17 CACHED
#18 [runtime 2/7] RUN addgroup -S ecotrack && adduser -S -G ecotrack -u 1001 ecotrack  && mkdir -p /app/logs && chown -R ecotrack:ecotra
ck /app
#18 CACHED
#19 [build 5/6] COPY src ./src
#19 CACHED
#20 [runtime 7/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/application/ ./
#20 CACHED
#21 naming to docker.io/library/ecotrack-api:local done

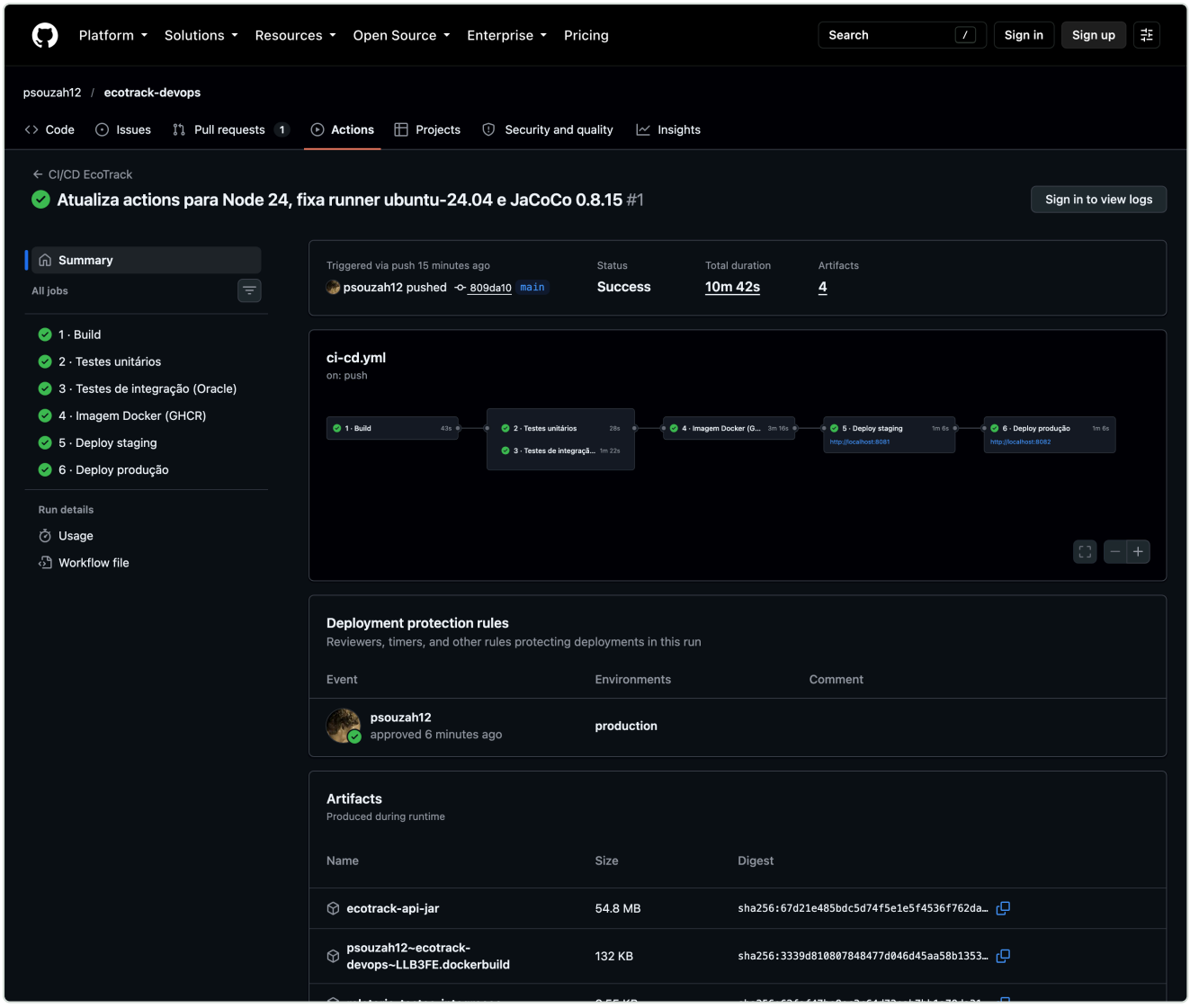
$ docker image ls ecotrack-api
REPOSITORY     TAG       IMAGE ID       SIZE
ecotrack-api   local     ffb071668c6f   408MB

$ docker image inspect ecotrack-api:local
Arquitetura: linux/arm64
Usuario:     ecotrack
Entrypoint:  ["java","org.springframework.boot.loader.launch.JarLauncher"]
Healthcheck: ["CMD-SHELL","wget -q -O /dev/null http://127.0.0.1:8080/actuator/health/readiness || exit 1"]
Porta:       {"8080/tcp":{}}
Versao:      1.1.0
Commit:      809da10084a5135c6ebf9403a0cc1571faf5e30e

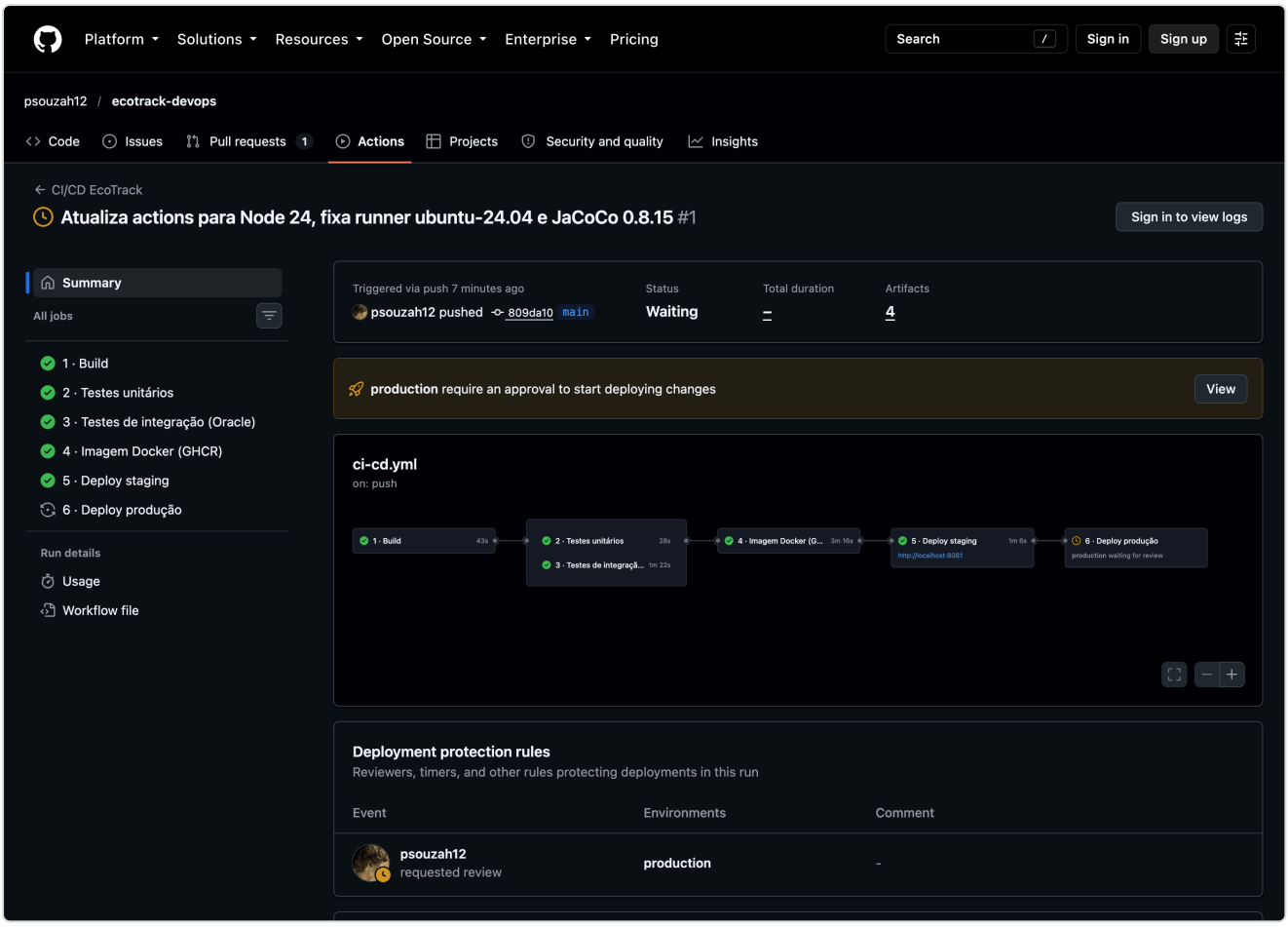
$ docker history ecotrack-api:local  (camadas)
SIZE      CREATED BY
0B        ENTRYPOINT ["java" "org.springframework.boot…
0B        HEALTHCHECK {Test:[CMD-SHELL wget -q -O /dev…
0B        EXPOSE [8080/tcp]
0B        USER ecotrack
0B        ENV APP_COMMIT=809da10084a5135c6ebf9403a0cc1…
0B        LABEL org.opencontainers.image.title=EcoTrac…
0B        ARG BUILD_DATE=2026-10-14T00:33:50Z
0B        ARG GIT_COMMIT=809da10084a5135c6ebf9403a0cc1…
0B        ARG APP_VERSION=1.1.0
352kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
4.1kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
688kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
63.5MB    COPY --chown=ecotrack:ecotrack /workspace/ta…
```

Build da imagem: estágios, camadas, usuário, healthcheck e metadados (versão e commit).

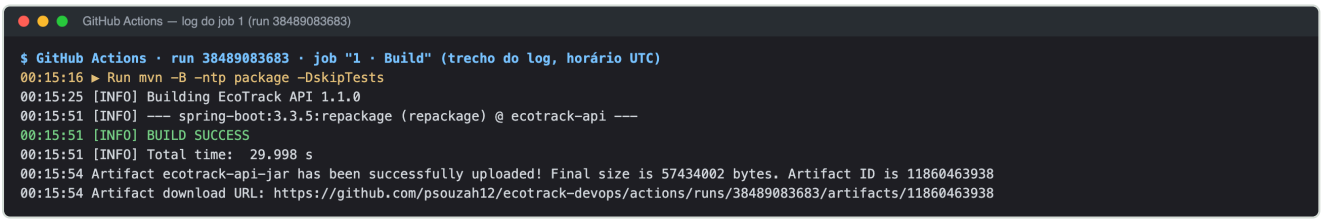
4. Pipeline em execução (build, testes e deploy)



Run #1 do workflow CI/CD EcoTrack, disparado pelo push do commit 809da10 em main: os 6 jobs passaram (10m42s, contando a espera pela aprovação de produção).



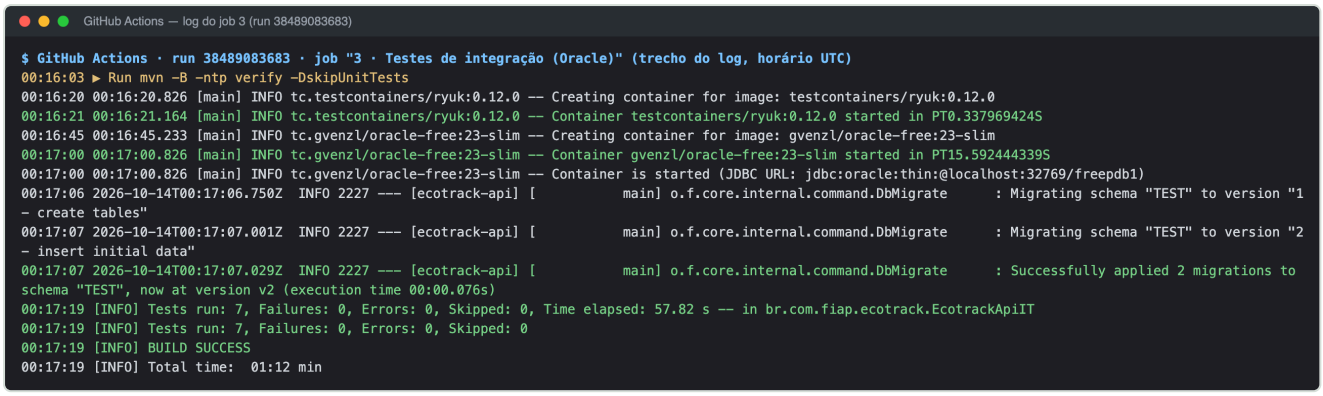
Depois do deploy em staging, o job de produção parou: o Environment production exige aprovação manual do revisor (psouzah12).



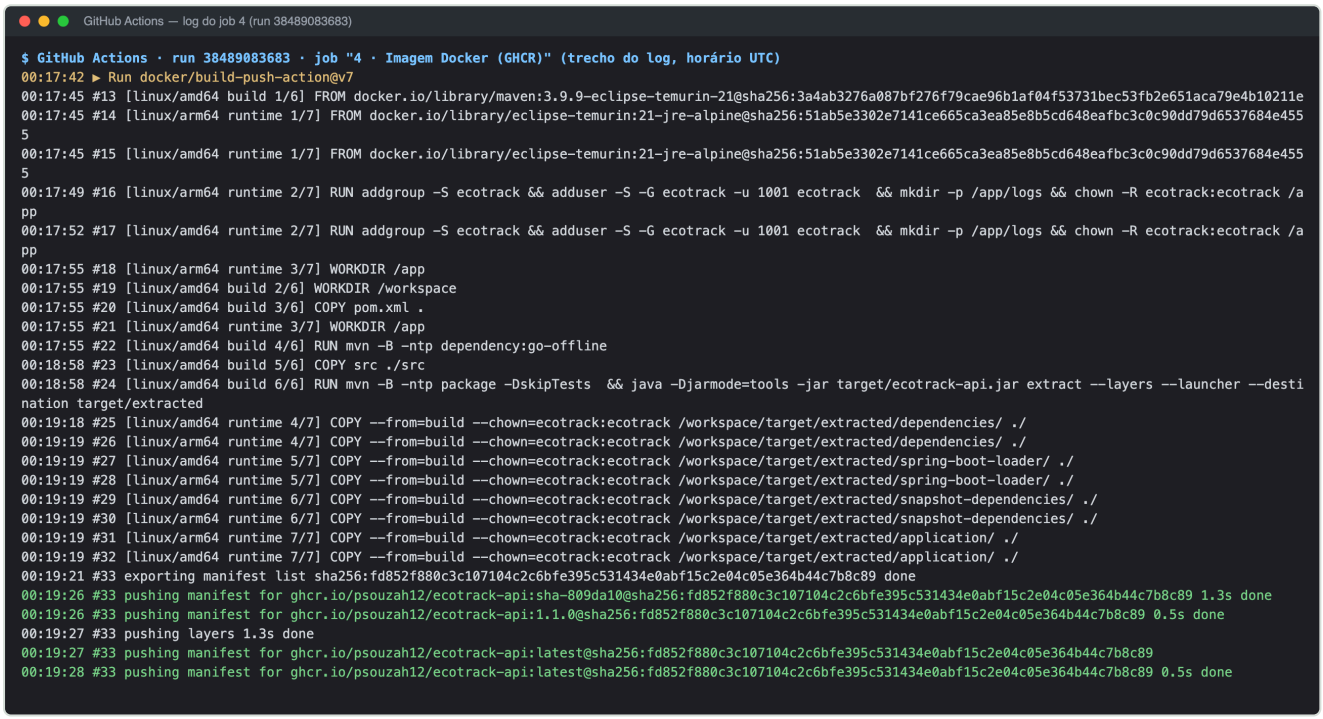
Job 1 · Build: trecho do log real do GitHub Actions (mvn package e upload do jar como artefato).



Job 2 · Testes unitários: 49 testes, 0 falhas, cobertura mínima verificada pelo JaCoCo.



Job 3 · Testes de integração: Testcontainers sobe o Oracle Free, o Flyway aplica as migrations e os 7 testes passam.



Job 4 · Imagem Docker: o estágio Maven roda só em linux/amd64, o estágio final é montado para amd64 e arm64, e as tags sha-809da10, 1.1.0 e latest são publicadas no GHCR.

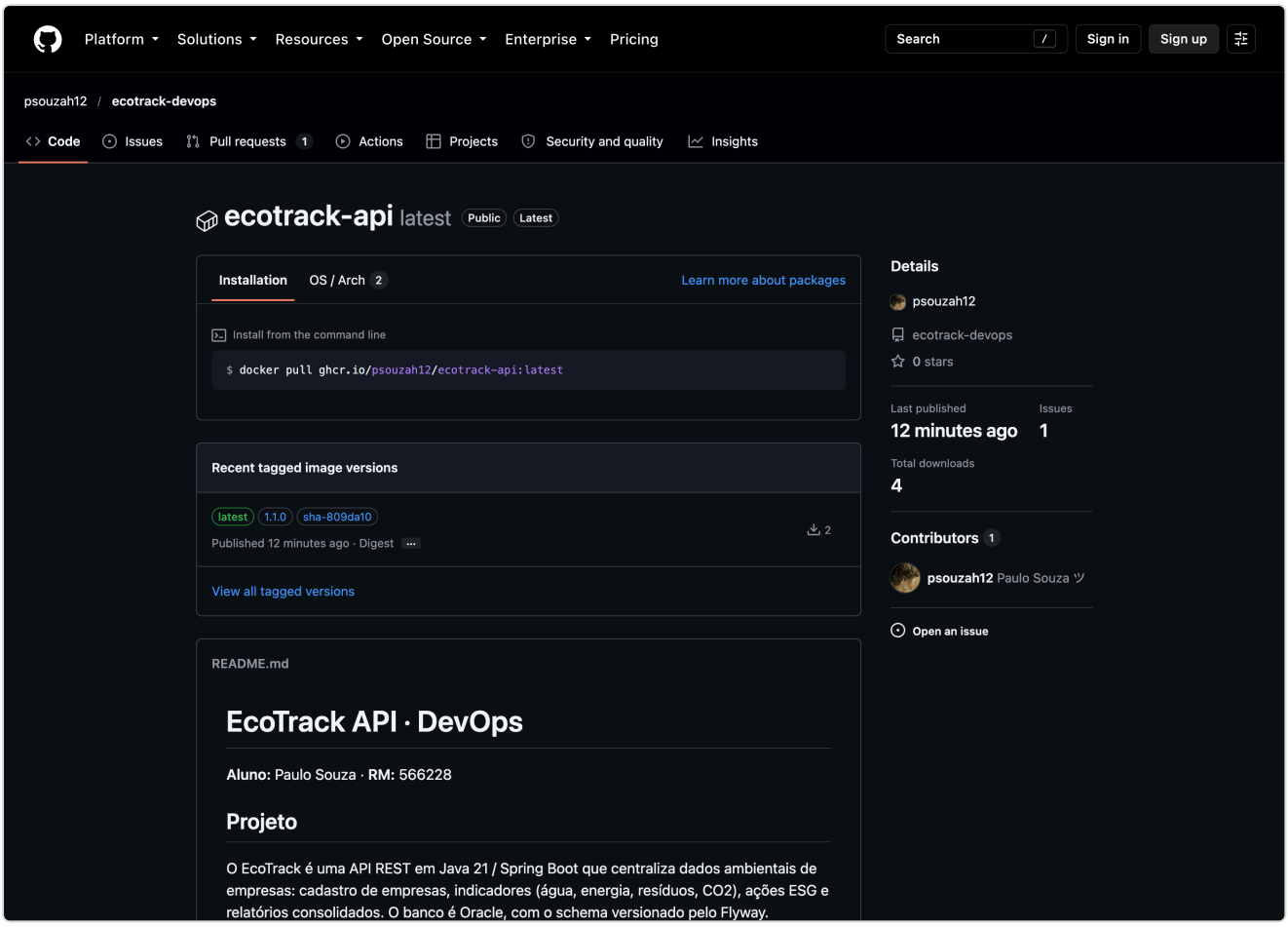
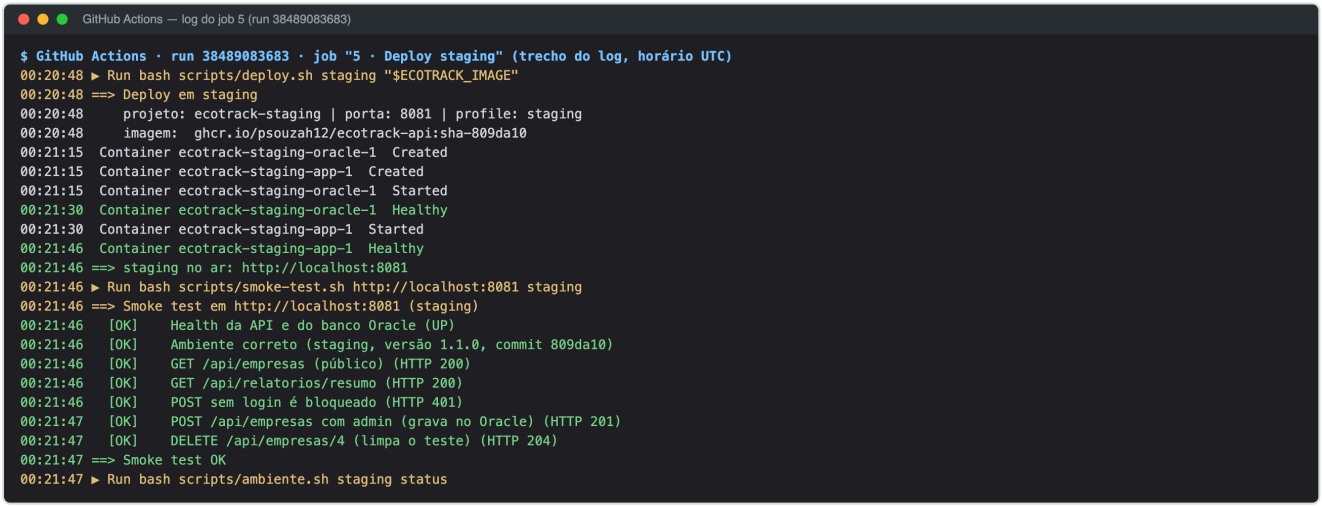
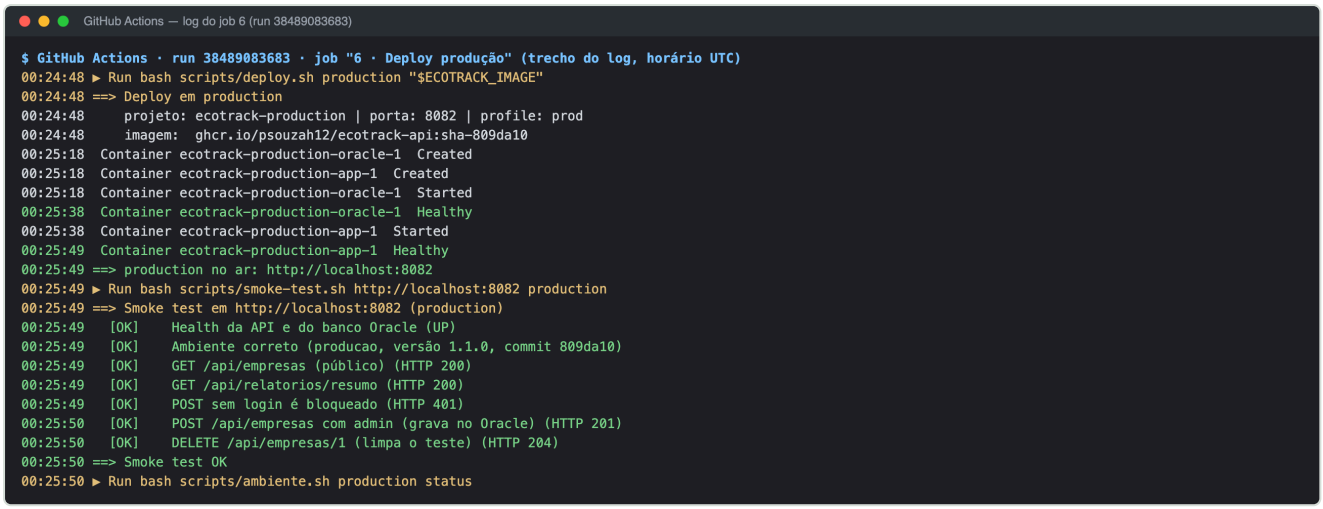


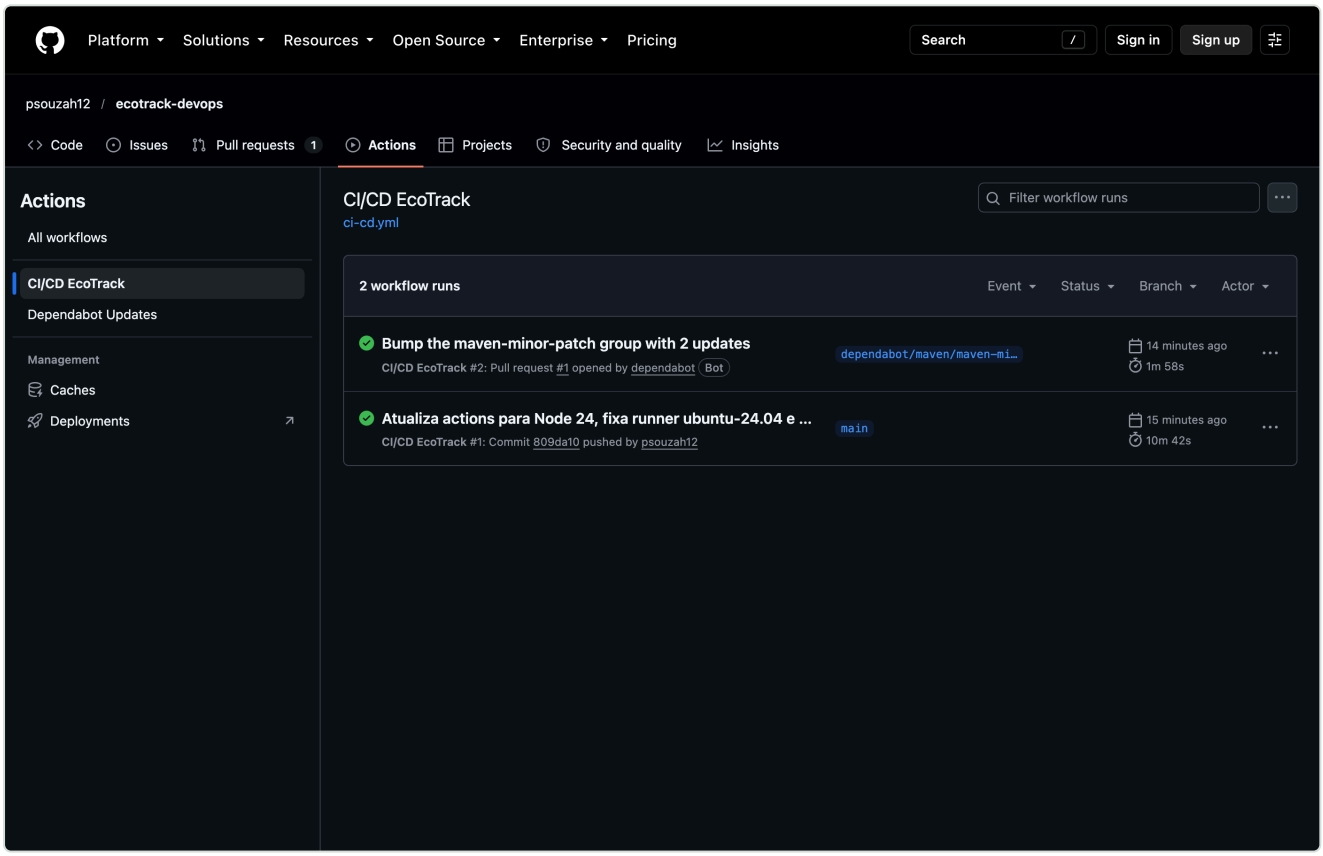
Imagem publicada no GitHub Container Registry (pública, 2 arquiteturas).



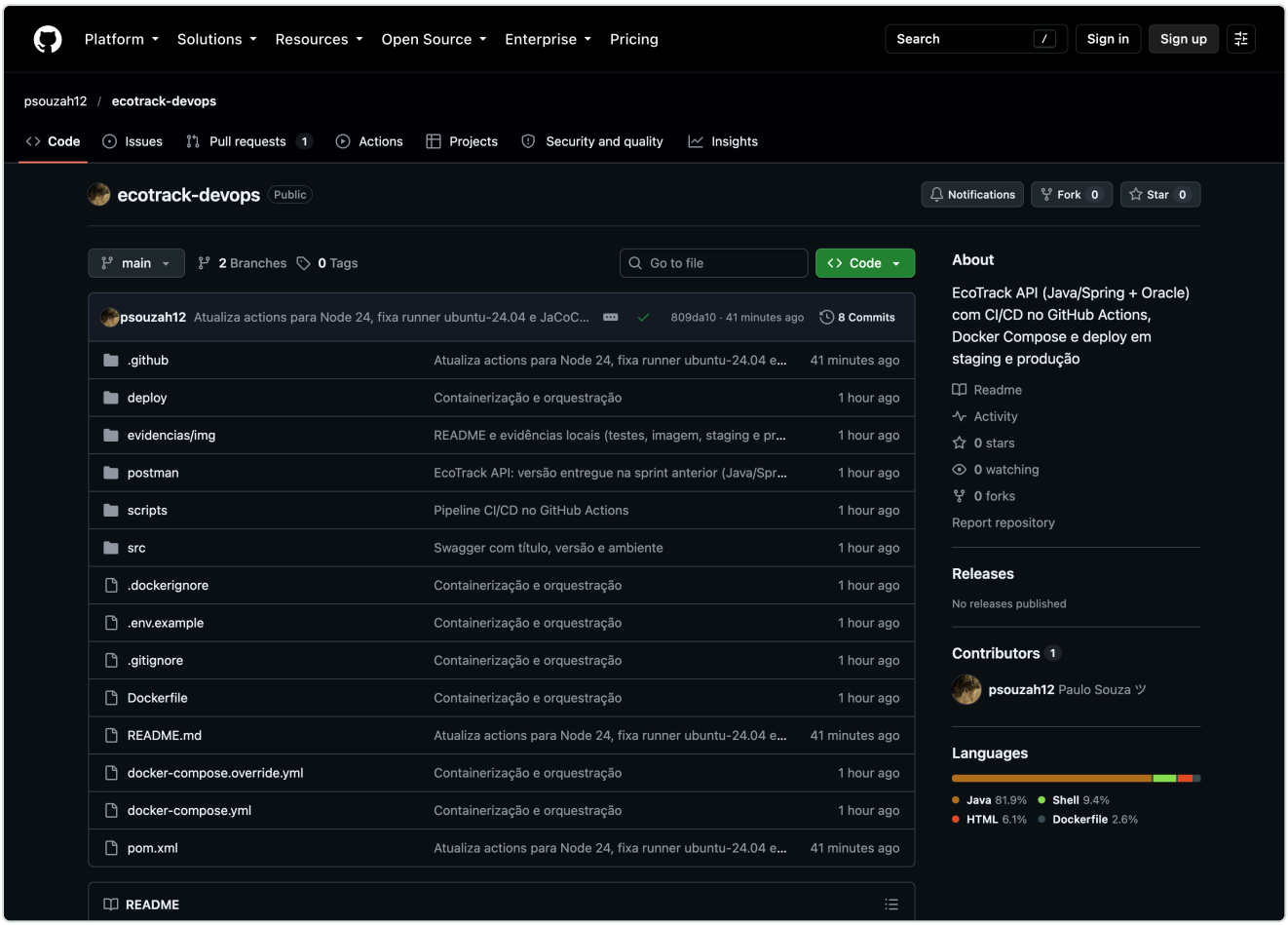
Job 5 · Deploy staging: deploy.sh com a imagem sha-809da10 e smoke test com 7 verificações OK.



Job 6 · Deploy produção (após aprovação): mesma imagem, mesmos scripts, smoke test OK.



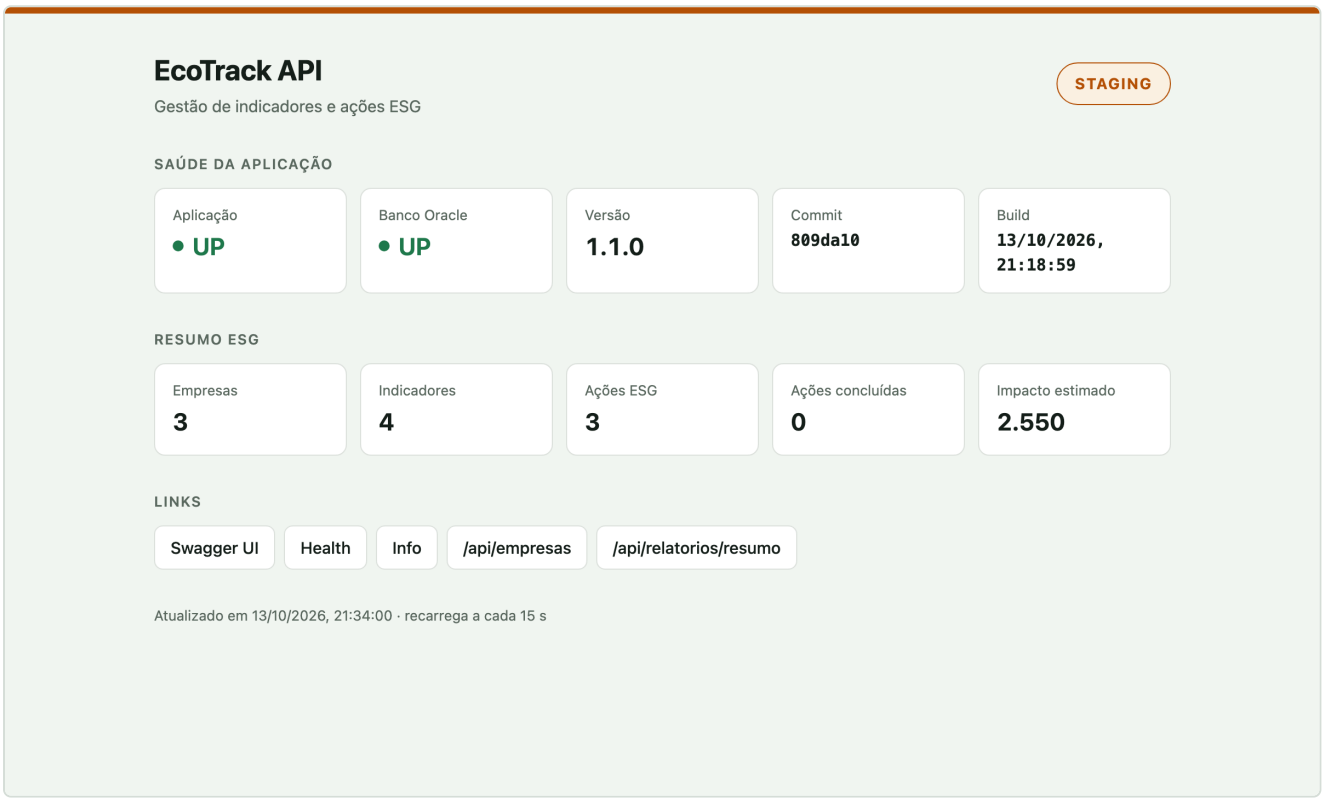
Histórico do workflow: execuções por push em main e pelos PRs do Dependabot, que também passam pelo pipeline antes de qualquer merge.



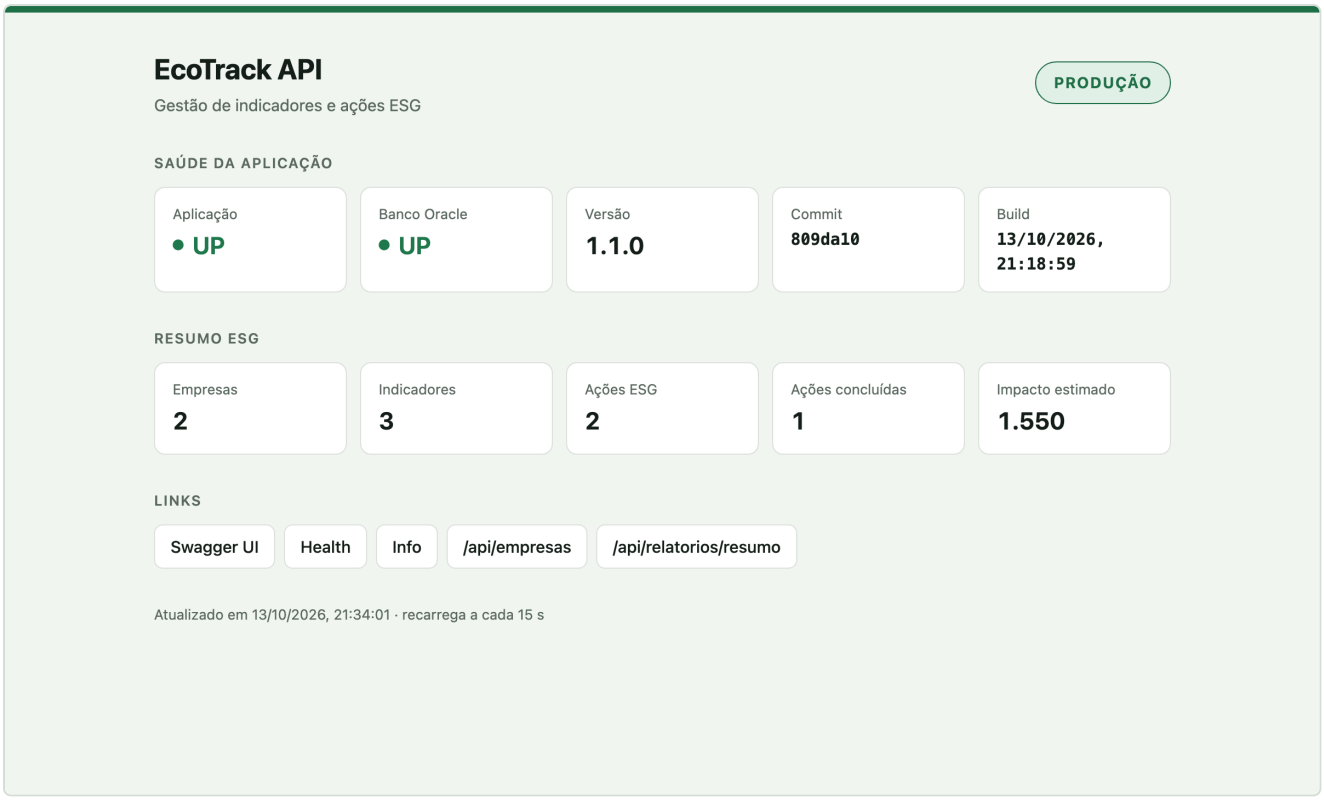
Repositório público com o código, o workflow e as evidências.

5. Ambientes staging e produção funcionando

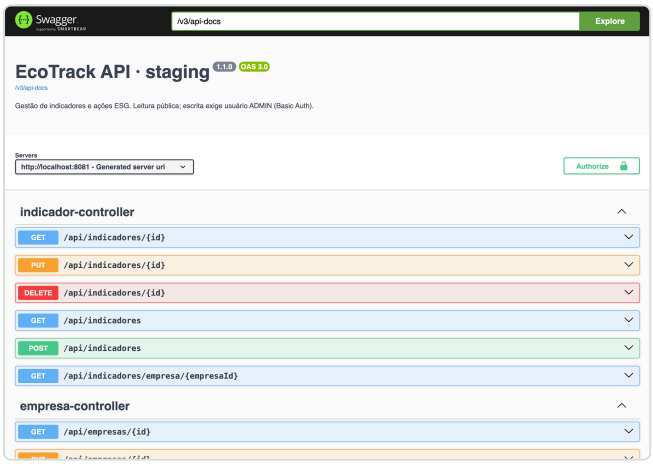
No pipeline, cada deploy sobe o ambiente completo dentro do runner do GitHub, valida com o smoke test e descarta tudo no fim (seção 4). Para ter os dois ambientes **permanentes**, os mesmos scripts foram executados numa máquina local com a **mesma imagem publicada pelo pipeline**, `ghcr.io/psouzah12/ecotrack-api:sha-809da10`. A página de status lê ao vivo `/actuator/info`, `/actuator/health` e `/api/relatorios/resumo`. Staging tem os dados de exemplo. Produção começou vazia; os registros foram cadastrados pela API com o usuário admin e continuaram no volume do Oracle depois do redeploy.



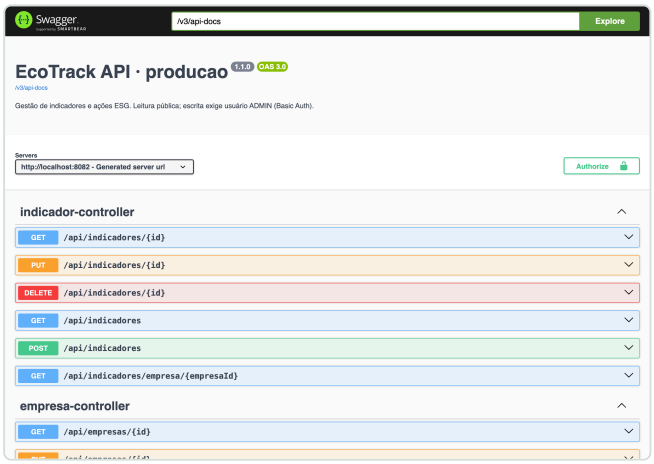
Staging (porta 8081): imagem sha-809da10 do GHCR, API e Oracle UP, dados de exemplo.



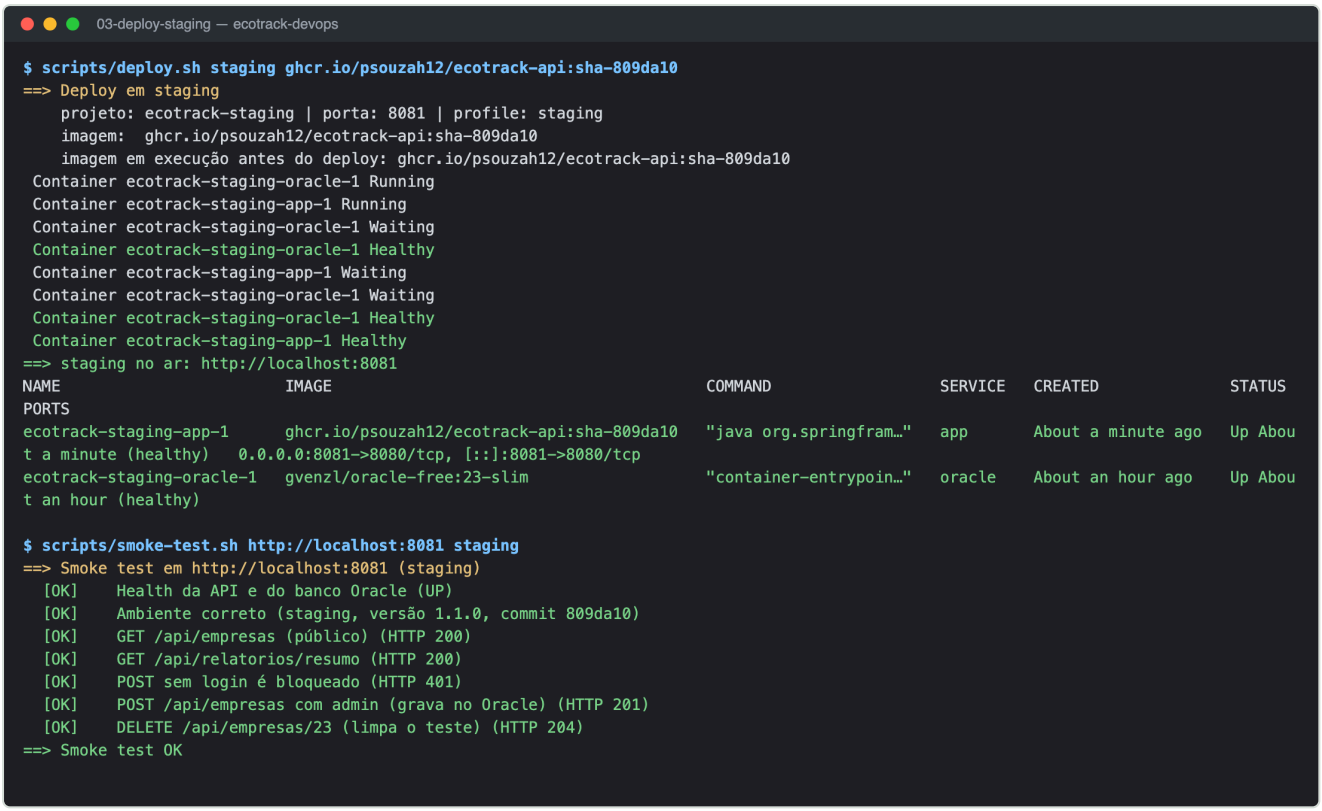
Produção (porta 8082): mesma imagem e commit validados em staging, banco próprio.



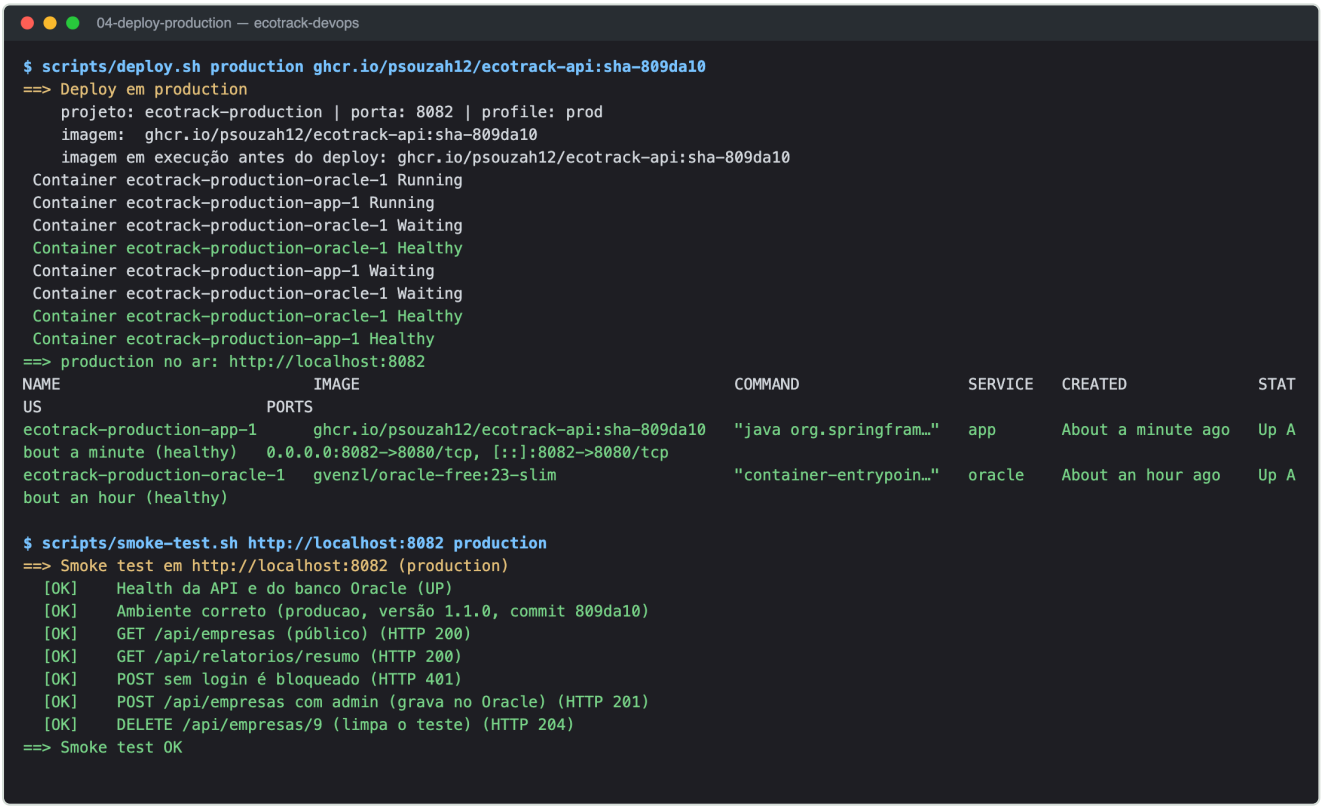
Swagger em staging.



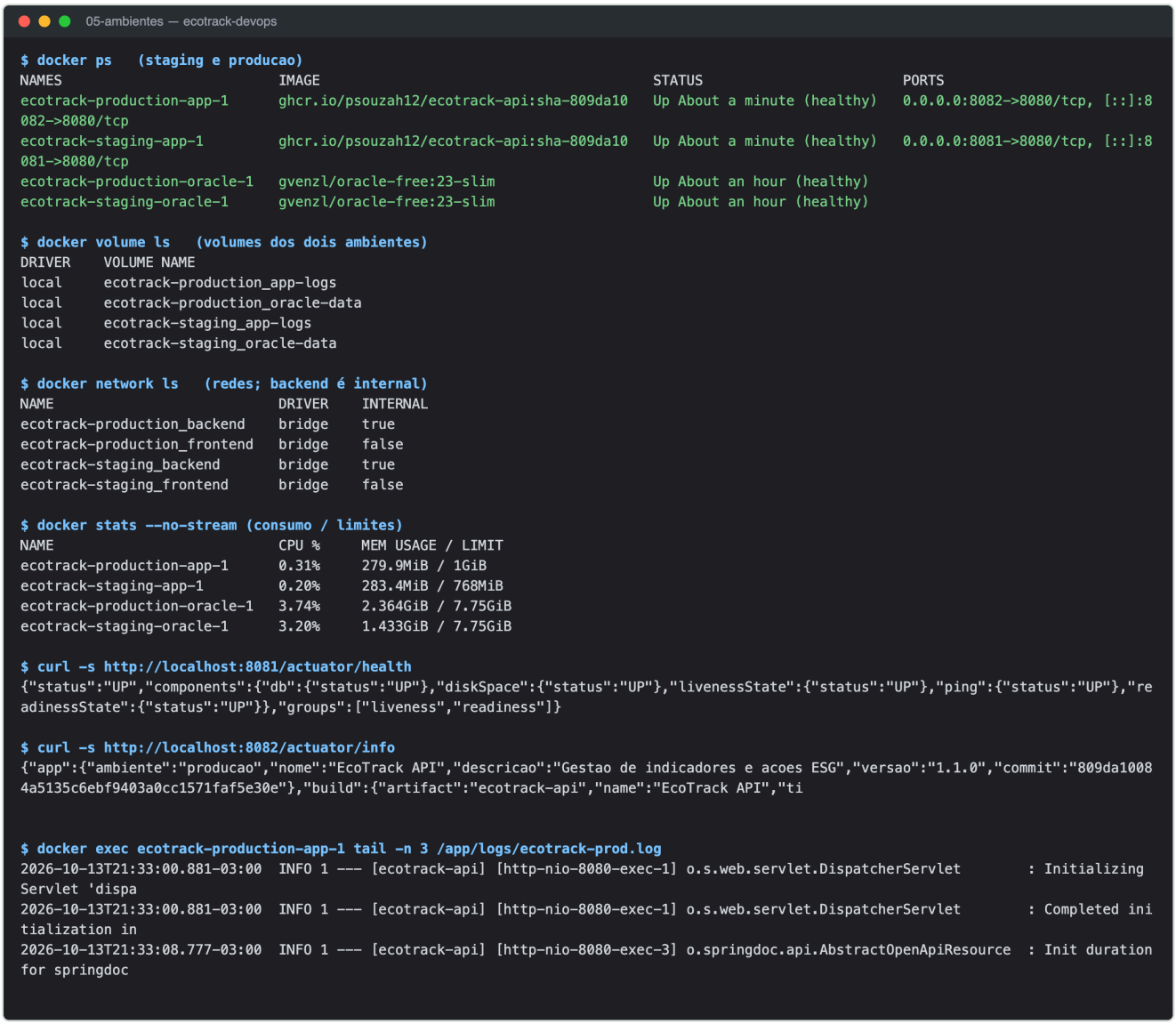
Swagger em produção.



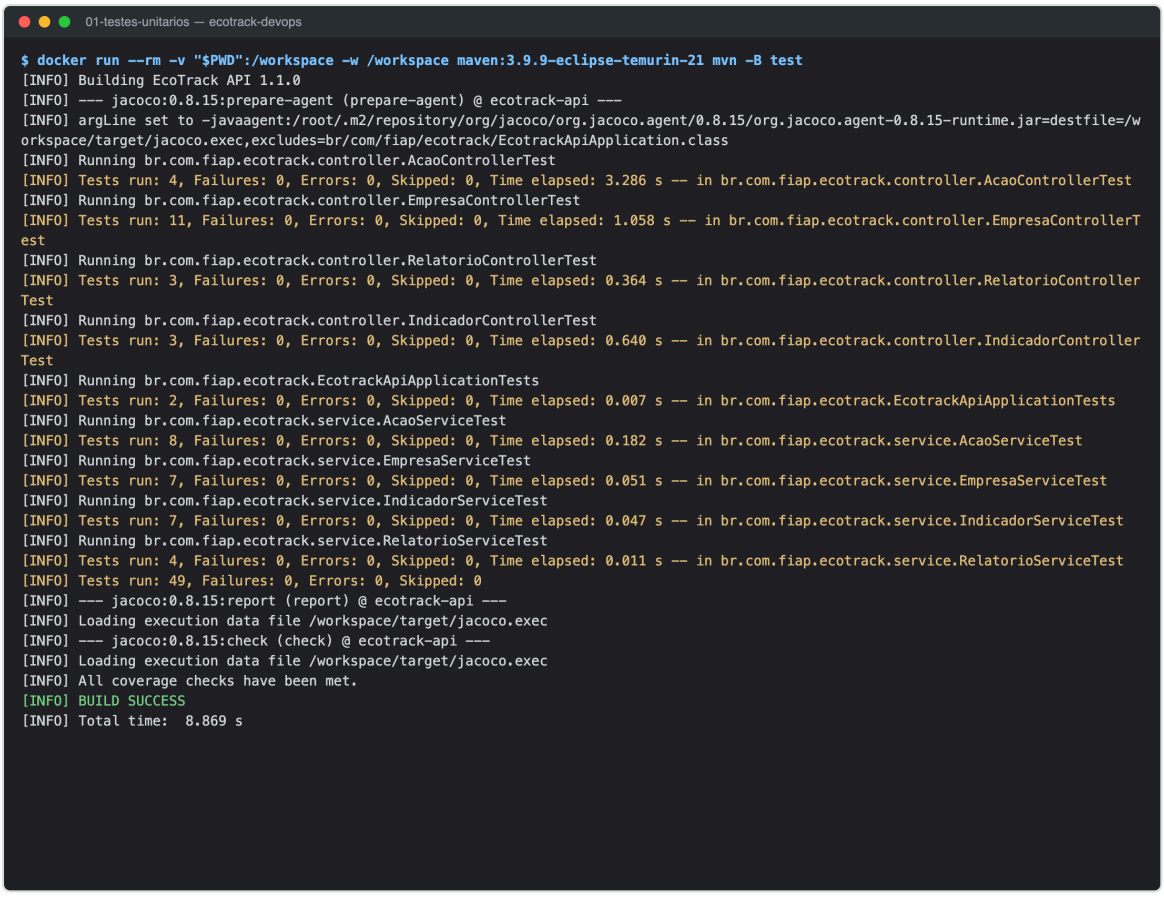
Deploy de staging com a imagem do GHCR (scripts/deploy.sh) e validação (scripts/smoke-test.sh).



Deploy de produção e smoke test.



Os dois ambientes: containers healthy, volumes, redes (backend interna), consumo de memória dentro dos limites e log da API gravado no volume.



Execução local dos testes unitários com o mesmo JDK 21 do pipeline, com verificação de cobertura.

6. Desafios encontrados e soluções

Testcontainers incompatível com o Docker Engine 29. O teste de integração não encontrava o Docker (*BadRequestException, Status 400*). O Docker 29 passou a recusar clientes com API anterior à 1.44, e o Testcontainers 1.19.8 trazido pelo Spring Boot 3.3.5 usava uma versão antiga. *Solução:* fixar `testcontainers.version=1.21.4` no `pom.xml`, compatível com o Engine 29.

LazyInitializationException ao desligar o open-in-view. Desligar o `spring.jpa.open-in-view` (boa prática em produção) fez listagens e o PATCH de status quebrarem com HTTP 500, porque a conversão para DTO acessava a empresa (LAZY) fora da sessão. Os testes unitários não pegaram; o teste de integração com Oracle real pegou. *Solução:* `@Transactional(readOnly = true)` nos services, `@Transactional` nos métodos de escrita, e um teste de integração específico para as listagens.

Erros 4xx viravam 500 sem log. O `@ExceptionHandler(Exception.class)` capturava tudo: rota inexistente, JSON mal formado e enum inválido viravam "Erro interno", sem nenhum registro em log. *Solução:* handlers específicos (400, 404, 405) e `log.error` no handler genérico, com testes cobrindo cada caso.

Caminho com espaço quebrava o deploy. O projeto fica em `Projeto ESG/`, e o comando do Compose guardado numa string (`$COMPOSE up`) era quebrado no espaço. *Solução:* trocar a string por uma função bash com os argumentos entre aspas.

Disco cheio durante o deploy. Ao subir o Oracle de staging, o disco do Mac chegou a 100%: o Docker Desktop guarda tudo num disco virtual que cresce no host, e cada Oracle ocupa cerca de 3 GB de dados. O Docker travou com erros de I/O. *Solução:* reiniciar o Docker, remover imagens antigas de outros projetos e o cache de build (≈ 10 GB liberados) e só então subir os dois ambientes. Lição: monitorar o espaço em disco do host de deploy.

Senha do Oracle só vale na criação do volume. A imagem do Oracle cria o usuário da aplicação apenas no primeiro start. Uma senha nova a cada deploy deixaria a API sem acesso ao banco. *Solução:* o `deploy.sh` reutiliza a senha já usada: vem dos secrets do GitHub ou, se não houver, é gerada uma única vez e guardada no host.

Mac (arm64) × runners do GitHub (amd64). A imagem gerada no pipeline precisa rodar nos dois. *Solução:* imagem multi-arquitetura com Buildx/QEMU. O estágio Maven roda em `$BUILDPLATFORM`, então o Java é compilado uma vez só e não sob emulação.

Cache do Maven no build da imagem. Um `RUN --mount=type=cache` para o `~/.m2` parece mais rápido, mas o cache de camadas do GitHub Actions não exporta cache mounts, e o pipeline baixaria todas as dependências a cada execução. *Solução:* camada própria com `dependency:go-offline`, que entra no cache `type=gha` e só é refeita quando o `pom.xml` muda.

O primeiro push não disparou o pipeline. O `paths-ignore` evita deploy quando só muda documentação. No primeiro push de um branch novo, o GitHub comparou apenas o último commit, que mexia só no README e nas evidências. *Solução:* o primeiro run foi disparado manualmente (`workflow_dispatch`), e os pushes seguintes com código passaram a disparar o pipeline sozinhos (evento `push`).

Avisos de depreciação no primeiro run. A primeira execução passou, mas gerou 9 avisos: as actions `checkout@v4`, `setup-java@v4` e outras rodavam em Node 20, descontinuado nos runners. *Solução:* atualizar as actions para as versões em Node 24 (checkout v7, setup-java v6, build-push v7…), conferindo nos release notes que nenhuma entrada usada mudou, e fixar o runner em `ubuntu-24.04`. As execuções seguintes terminaram sem avisos.

Dependabot abriu 10 PRs, inclusive versões major. Logo depois do primeiro push vieram PRs para Spring Boot 4, springdoc 3, Java 25 etc. O pipeline rodou em cada PR e reprovou o Spring Boot 4 e o springdoc 3, ou seja, o CI barrou dependências incompatíveis. *Solução:* configurar o Dependabot para ignorar versões major (que exigem migração planejada) e agrupar minor/patch. Com isso, 9 dos 10 PRs foram fechados automaticamente.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✔
Dockerfile funcional	✔
docker-compose.yml ou arquivos Kubernetes	✔
Pipeline com etapas de build, teste e deploy	✔
README.md com instruções e prints	✔
Documentação técnica com evidências (PDF ou PPT)	✔
Deploy realizado nos ambientes staging e produção	✔